

UNIVERSIDAD NACIONAL DEL COMAHUE
Centro Universitario Regional Zona Atlantica

Tecnicatura Universitaria en Administracion de Sistemas y Software Libre

Trabajo Practico 2

Scripts avanzados en Bash

Dato	Informacion
Asignatura	Automatizacion y Scripting
Alumno	Ulises Espinosa
Legajo	CURZA-4047
Fecha	21 de septiembre de 2026

Repositorio: github.com/Nikcodra/curza-automatizacion-y-scripting

Indice

Indice	2
1. Introduccion	3
1.1 Alcance y entregables	3
2. Enunciado del problema	3
2.1 Buscador de versiones en PDF	3
2.2 Renombrador y organizador automatico	3
2.3 Monitoreo interactivo de recursos	3
2.4 Verificador de sitios web	3
2.5 Respalador con prevencion de ejecucion simultanea	3
2.6 Estructura solicitada	4
3. Hipotesis, supuestos y decisiones	4
4. Resolucion	4
4.1 pdf_version.sh	4
4.2 organizador.sh	4
4.3 monitorear.sh	5
4.4 chequear_paginas.sh	5
4.5 backup_manager.sh	5
5. Pruebas y resultados	6
6. Repositorio e historial	6
7. Dificultades encontradas	7
8. Conclusion	7

1. Introduccion

El trabajo desarrolla cinco herramientas de administracion en Bash para buscar metadatos de archivos PDF, organizar archivos, consultar recursos del sistema, verificar sitios web y generar respaldos protegidos contra ejecuciones simultaneas. Los scripts se probaron en Ubuntu 24.04 sobre WSL 2 y se conservaron los resultados requeridos en el repositorio.

La implementacion utiliza solamente comandos disponibles en Ubuntu y las estructuras estudiadas en las clases 4, 5 y 6: condicionales, bucles, arreglos, sustitucion de cadenas, funciones, select, case, find, curl, trap y archivos de bloqueo.

1.1 Alcance y entregables

- Cinco scripts ejecutables dentro de codigo/.
- Archivo sitios_CURZA-4047.txt con cuatro URLs de prueba.
- Log chequeo_CURZA-4047.log con codigos 200, 302, 404 y 500.
- Respaldo comprimido backup_CURZA-4047_20260921_231841.tar.gz.
- README con instrucciones, resultados y capturas reales de terminal.
- Repositorio Git con avances separados y mensajes claros.

2. Enunciado del problema

Como administrador de la red de laboratorios del CURZAS, se solicita automatizar la gestion y el monitoreo diario de servidores de archivos. La solucion debe procesar archivos, consultar servicios web, ordenar directorios y evitar colisiones durante respaldos.

2.1 Buscador de versiones en PDF

Crear pdf_version.sh para recorrer el directorio actual y sus subdirectorios, localizar archivos con extension .pdf, obtener la primera linea mediante head -n 1 e imprimir el archivo y la version PDF. Deben omitirse los nombres que contengan la palabra excluir o las iniciales del alumno.

2.2 Renombrador y organizador automatico

Crear organizador.sh. Debe recibir un directorio, comprobar que exista y sea accesible, y terminar con codigo 1 ante una entrada invalida. Debe crear imagenes/, documentos/, comprimidos/ y otros/, mover cada archivo segun su extension y transformar los nombres terminados en .old para que finalicen en .backup.

2.3 Monitoreo interactivo de recursos

Crear monitorear.sh con un menu select/case que permita consultar memoria RAM en MB, mostrar los cinco archivos mayores a 10 MB en HOME, consultar el espacio de las particiones montadas y salir con un saludo que incluya el legajo. El menu debe repetirse hasta elegir Salir.

2.4 Verificador de sitios web

Crear chequear_paginas.sh para recibir URLs como argumentos o leerlas desde sitios_CURZA-4047.txt. Cada URL debe consultarse con curl, mostrarse en verde para 200, amarillo para 3xx y rojo para 4xx o 5xx, y guardarse sin codigos de color en logs/chequeo_CURZA-4047.log.

2.5 Respalador con prevencion de ejecucion simultanea

Crear backup_manager.sh. Debe adquirir de manera atomica /var/lock/backup_CURZA-4047.lock mediante mkdir, terminar con codigo 9 si el bloqueo ya existe y registrar con trap la liberacion del bloqueo. Luego debe copiar los archivos del directorio de codigo modificados durante las ultimas 24 horas a /tmp/backup_CURZA-4047/ y generar un tar.gz fechado dentro de logs/.

2.6 Estructura solicitada

```
tp2-espinosa-CURZA-4047/  
|-- README.md  
|-- sitios_CURZA-4047.txt  
|-- codigo/  
|   |-- pdf_version.sh  
|   |-- organizador.sh  
|   |-- monitorear.sh  
|   |-- chequear_paginas.sh  
|   |-- backup_manager.sh  
|-- logs/  
|   |-- chequeo_CURZA-4047.log  
|   |-- backup_CURZA-4047_FECHA.tar.gz
```

3. Hipotesis, supuestos y decisiones

- La expresion iniciales del alumno se implementa como ue, sin distinguir mayusculas y minusculas.
- El organizador procesa los archivos regulares ubicados directamente en el directorio recibido. No vuelve a recorrer las carpetas de clasificacion que el mismo crea.
- Un archivo .old se renombra a .backup y luego se mueve a otros/, porque .backup no pertenece a las categorias expresamente indicadas.
- Ante nombres repetidos, el organizador agrega un numero al destino para no sobrescribir datos existentes.
- La opcion de RAM muestra los valores actuales de free -m cada vez que el usuario la selecciona.
- El verificador no sigue redirecciones, porque necesita conservar el codigo 3xx para mostrarlo en amarillo.
- Los errores de conexion de curl se representan con codigo 000 y se muestran como Sin respuesta.
- backup_manager.sh se ejecuta con un usuario autorizado para crear directorios en /var/lock.
- La prohibicion de abortos se interpreta como ausencia de fallos inesperados. Se mantienen las salidas controladas 1 y 9 exigidas por el propio enunciado.
- Aunque el enunciado menciona capturas de tres scripts, se documentan los cinco para cubrir todas las tareas solicitadas.

4. Resolucion

4.1 pdf_version.sh

El script utiliza find con terminacion nula para admitir rutas con espacios. Cada nombre se convierte temporalmente a minusculas para aplicar los dos filtros. La primera linea se obtiene con head -n 1 y una expresion regular recupera el numero que sigue a %PDF-.

```
while IFS= read -r -d '' archivo; do  
    nombre="${archivo##*/}"  
    nombre_minusculas="${nombre,,}"  
    [[ "$nombre_minusculas" == *excluir* || "$nombre_minusculas" == *ue* ]] && continue  
    primera_linea=$(LC_ALL=C head -n 1 -- "$archivo" 2>/dev/null)  
done < <(find . -type f -iname '*.pdf' -print0)
```

La prueba incluyo manual.pdf, excluir_manual.pdf, UE_documento.pdf y sub/anexo.pdf. Solo manual.pdf y sub/anexo.pdf fueron informados, ambos con version 1.4.

4.2 organizador.sh

Primero se valida la cantidad de argumentos y los permisos efectivos del directorio. Luego se crean las cuatro carpetas y se recorre exclusivamente el primer nivel. Un case sobre el nombre en minusculas resuelve las categorias y detecta correctamente la extension compuesta .tar.gz.

```

case "$nombre_minusculas" in
*.jpg|*.png)      categoria="imagenes" ;;
*.pdf|*.txt|*.docx) categoria="documentos" ;;
*.zip|*.tar.gz|*.rar) categoria="comprimidos" ;;
*)                categoria="otros" ;;
esac

```

La prueba clasifico foto.jpg, nota.txt, documento.pdf, paquete.tar.gz, legado.old y programa.bin. El resultado dejo legado.backup dentro de otros/ y los cinco elementos restantes en sus categorias. Tambien se comprobe el codigo de salida 1 frente a una ruta inexistente.

4.3 monitorear.sh

El menu se implementa con un arreglo de cuatro opciones, select y case. Las operaciones estan separadas en funciones sencillas. La busqueda de archivos grandes usa find -size +10M, ordenamiento numerico y head -n 5. El menu solo termina al seleccionar Salir.

```

select opcion in "${opciones[@]}"; do
  case "$opcion" in
    "Monitorear memoria RAM") mostrar_memoria ;;
    "Buscar archivos grandes") buscar_archivos_grandes ;;
    "Espacio en particiones") mostrar_particiones ;;
    "Salir") echo "Hasta luego. Ulises Espinosa - CURZA-4047."; break ;;
    *) echo "Opcion invalida: $REPLY." ;;
  esac
done

```

Se probaron las cuatro opciones en Ubuntu. Para la busqueda se creo temporalmente un archivo de 12 MB; el script lo detecto y mostro su tamano como 12.00 MB.

4.4 chequear_paginas.sh

Si existen argumentos, el arreglo urls conserva cada parametro por separado. En caso contrario se lee sitios_CURZA-4047.txt linea por linea. curl se ejecuta de forma silenciosa, descarta el cuerpo y devuelve unicamente el codigo HTTP. Los colores solo se aplican a la terminal; el log queda en texto plano.

```

codigo=$(curl -s -o /dev/null -w '%{http_code}' --connect-timeout 10 --max-time 20 "$url")

case "$codigo" in
200) color="$VERDE"; estado="OK" ;;
3??) color="$AMARILLO"; estado="REDIRECCION" ;;
4??|5??) color="$ROJO"; estado="ERROR HTTP" ;;
*) color="$ROJO"; estado="SIN RESPUESTA" ;;
esac

```

URL	Codigo	Resultado
httpbin.org/status/200	200	OK
httpbin.org/redirect/1	302	REDIRECCION
httpbin.org/status/404	404	ERROR HTTP
httpbin.org/status/500	500	ERROR HTTP

4.5 backup_manager.sh

El bloqueo se adquiere antes de crear archivos temporales. El trap se registra solamente despues de que mkdir tiene exito; de ese modo, una segunda instancia no puede borrar el bloqueo perteneciente a la primera. find selecciona archivos modificados durante las ultimas 24 horas y cp --parents conserva su estructura relativa dentro del directorio temporal.

```

if ! mkdir "/var/lock/backup_CURZA-4047.lock" 2>/dev/null; then
    echo "Error: ya existe una ejecucion de backup_manager.sh." >&2
    exit 9
fi
trap 'rmdir "/var/lock/backup_CURZA-4047.lock"' EXIT

find . -mtime -1 -type f -print0
tar -czf "$archivo_backup" -C "$TEMP_DIR" .

```

La ejecucion normal genero backup_CURZA-4047_20260921_231841.tar.gz con los cinco scripts. Despues se creo manualmente el directorio de bloqueo y se ejecuto una segunda instancia: el codigo observado dentro de Ubuntu fue 9. Al finalizar, el directorio de bloqueo no permanecio en /var/lock.

5. Pruebas y resultados

Comprobacion	Resultado	Estado
Sintaxis Bash de los cinco scripts	bash -n sin errores	Correcto
Permisos de ejecucion	Los cinco archivos estan registrados como ejecutables	Correcto
Filtros del buscador PDF	Se omitieron excluir y UE	Correcto
Clasificacion de seis extensiones	Todas quedaron en el directorio previsto	Correcto
Directorio invalido	organizador.sh devolvio codigo 1	Correcto
Cuatro opciones del monitor	RAM, archivo de 12 MB, particiones y salida	Correcto
Estados HTTP	200, 302, 404 y 500 registrados	Correcto
Contenido del tar.gz	Incluye los cinco scripts	Correcto
Segunda instancia del respaldo	Finalizo con codigo 9	Correcto

La auditoria se ejecuto de principio a fin en Ubuntu 24.04 y concluyo con el mensaje AUDITORIA COMPLETA: OK. Los datos temporales usados para las pruebas fueron eliminados al terminar.

6. Repositorio e historial

El codigo se encuentra en el repositorio privado github.com/Nikcodra/curza-automatizacion-y-scripting. La carpeta correspondiente a este trabajo es tp2-espinosa-CURZA-4047/.

Avance	Mensaje registrado
1	Preparar estructura del trabajo practico 2
2	Agregar buscador de PDF y organizador de archivos
3	Incorporar monitoreo interactivo y chequeo web
4	Implementar respaldo con bloqueo de ejecucion

7. Dificultades encontradas

- La primera URL de prueba elegida no respondio desde WSL. Se sustituyo por <httpbin.org/status/200> para conservar un caso 200 verificable.
- La extension .tar.gz no puede clasificarse observando solamente el texto posterior al ultimo punto; por eso se compara el nombre completo mediante patrones case.
- El trap del respaldo no debe registrarse antes de adquirir el bloqueo, porque una instancia rechazada podria eliminar el directorio perteneciente a otra ejecucion.
- Las pruebas automatizadas de menus requieren enviar tambien la confirmacion para volver al menu, ademas del numero de cada opcion.

8. Conclusion

Los cinco scripts cumplen las tareas solicitadas y fueron probados en un entorno Ubuntu. La solucion mantiene validaciones visibles, protege los archivos frente a sobrescrituras, conserva los errores HTTP en un log legible y evita respaldos simultaneos mediante una operacion atomica. El repositorio se organizo por trabajo practico y conserva un historial separado por avances funcionales.